Static meta-programming (AST transformations)





Václav Pech

NPRG014 2026/2027

http://www.vaclavpech.eu

@vaclav_pech

Last time agenda



Dynamic method call dispatch
Dynamic object creation
Categories
Builders
Internal DSLs
LLM-backed semantic primitives

Part 5

Static meta-programming

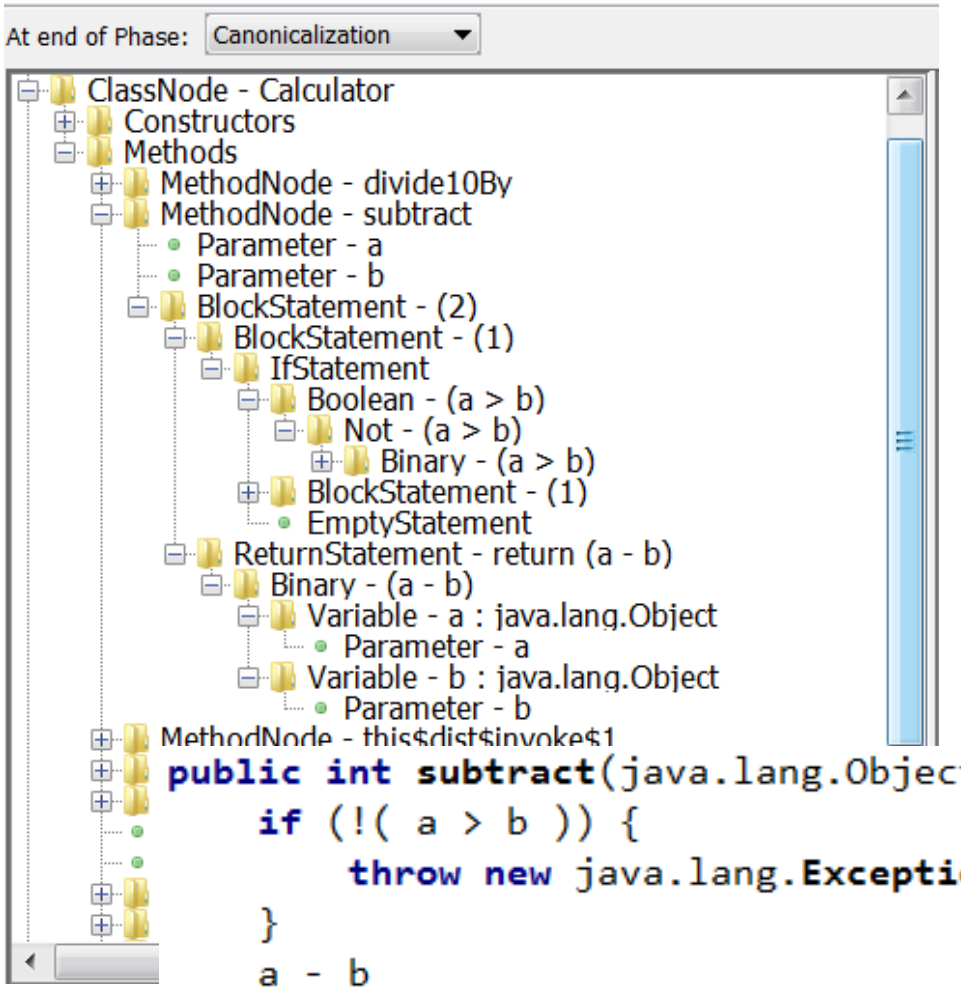
(program modifications at compile-time
aka compiler plugins)

Is determinism overrated?

```
130
131 class Calculator {
132     def plus(a, b) {
133         a + b
134     }
135
136     @VibeCoded("Subtracts the parameter b from parameter a")
137     def minus(a, b) {}
138
139     @VibeCoded("Divides parameters a and b, if b is 0 throw IllegalArgumentException")
140     def divide(a, b) {}
141
142     @VibeCoded("Multiplies parameters a and b")
143     def multiply(a, b) {}
144
145     @VibeCoded("Calculates a factorial of n")
146     def factorial(n) {}
147 }
148
149 println 'Result: ' + new Calculator().factorial(6)
150
151
```

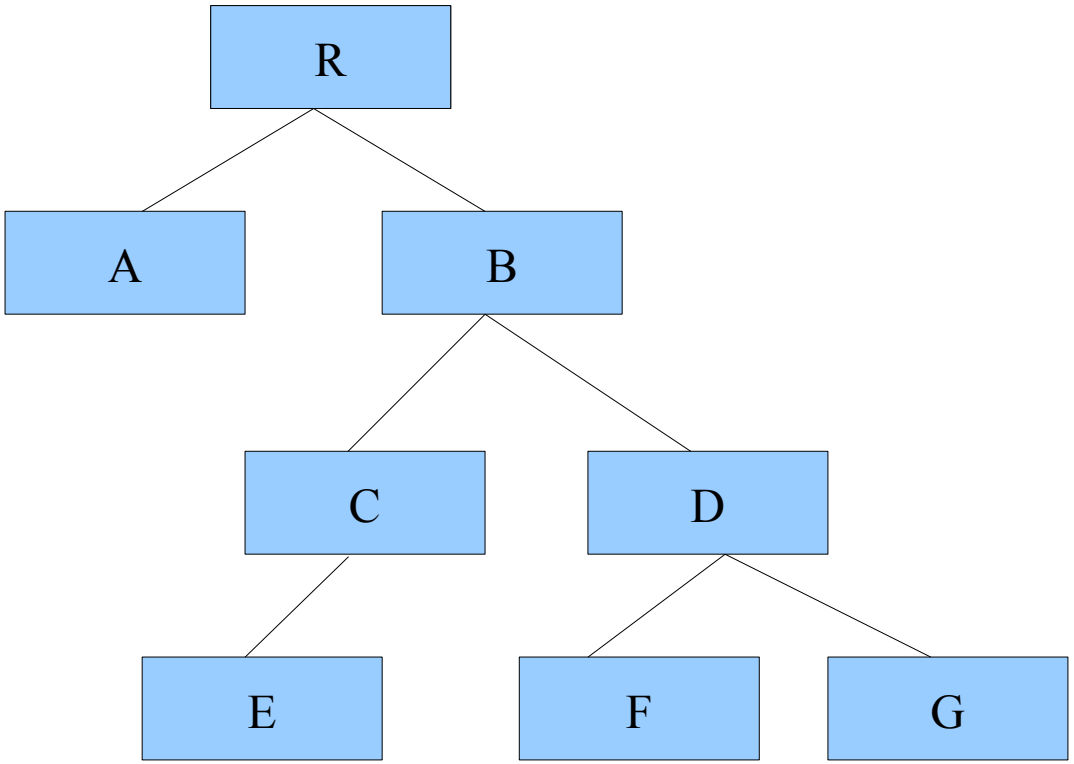
```
Result: 720
```

AST




```
public int subtract(java.lang.Object a, java.lang.Object b) {
    if (!( a > b )) {
        throw new java.lang.Exception('Precondition violated: {a > b}')
    }
    a - b
}
```

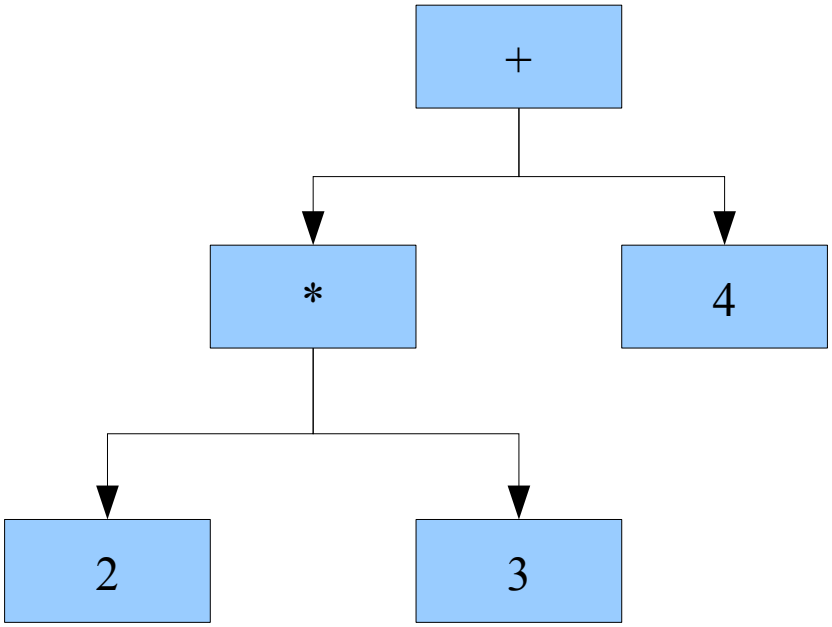
Programs are trees



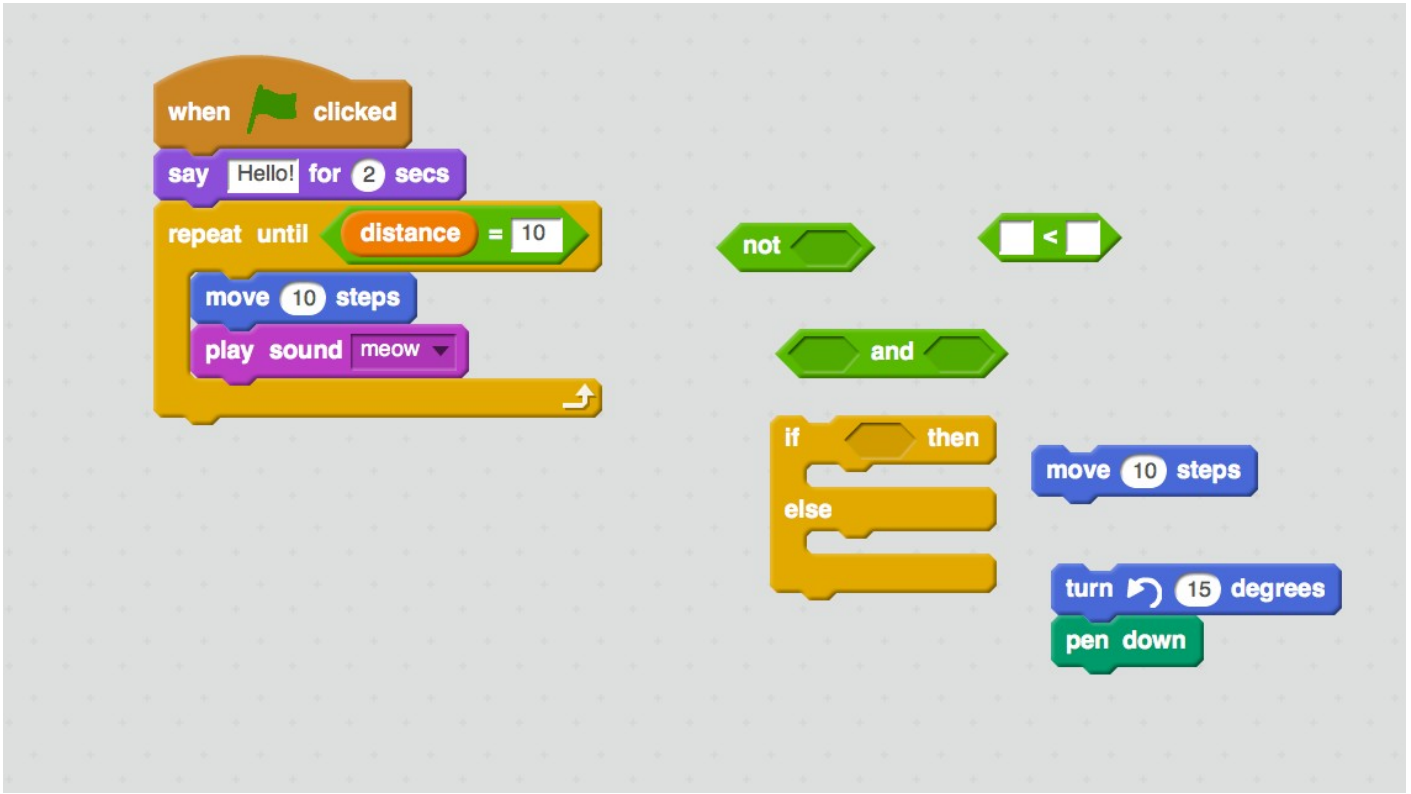


Lisp (1958)

(+ (* 2 3) 4)






when clicked
say Hello! for 2 secs
repeat until distance = 10
move 10 steps
play sound meow
not
<
and
if then
else
move 10 steps
turn 15 degrees
pen down

```
public class Demo {

  private static void foo() {
    System.out.println("Foo called");
  }

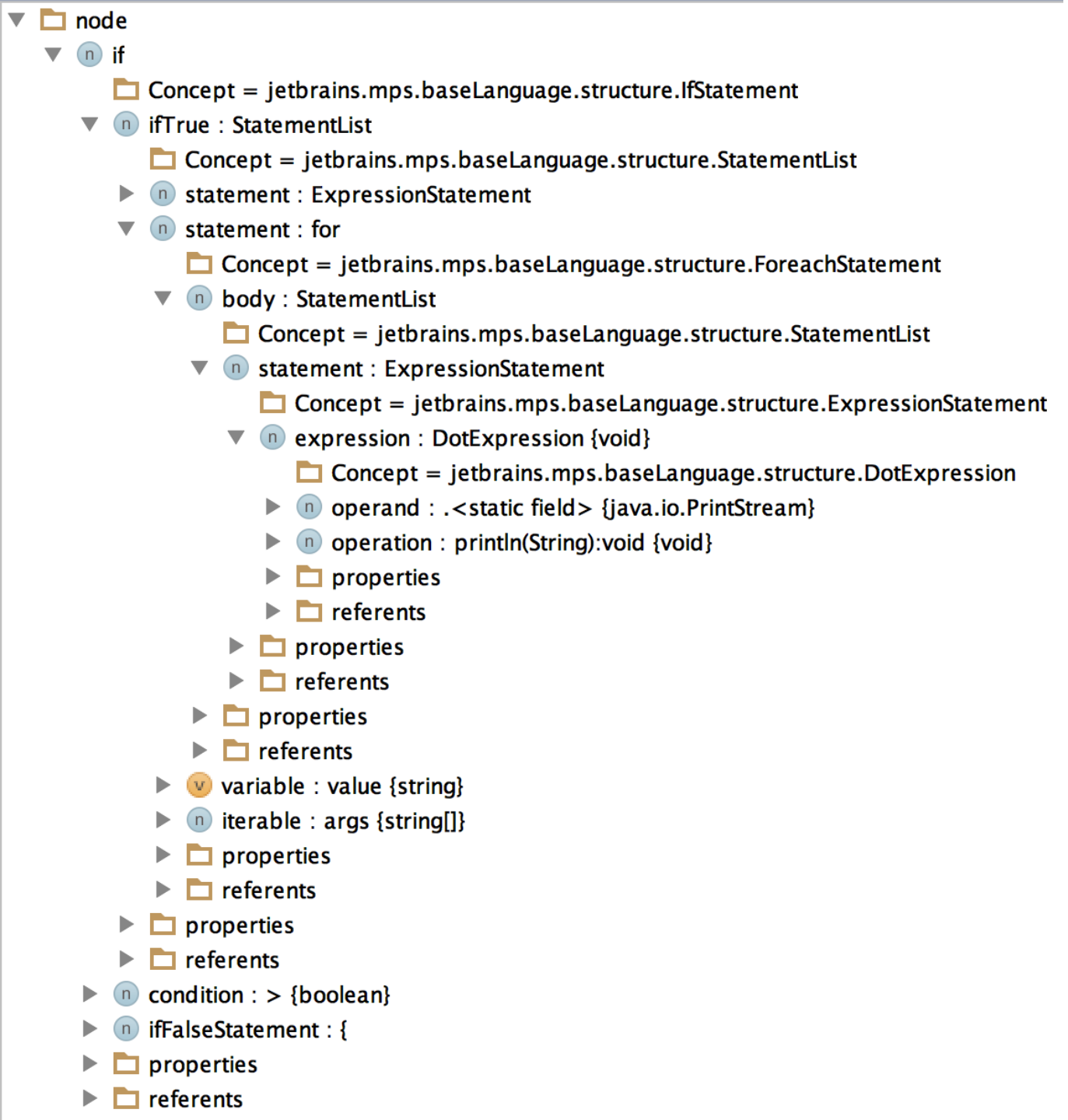
  public static void main(string[] args) {

    System.out.println("Application started");

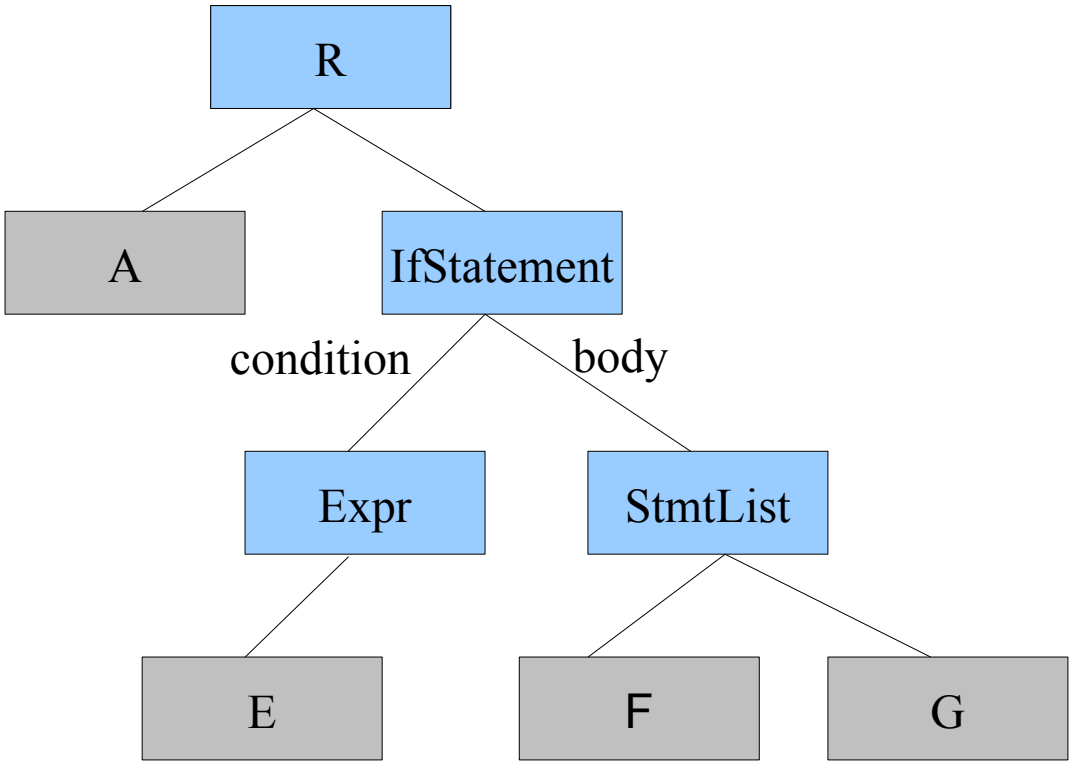
    if (args.length > 0) {
      System.out.println("Supplied arguments");
      for (string value : args) {
        System.out.println("Argument: " + value);
      }
    } else {
      System.out.println("No arguments provided");
    }

    foo();
    System.out.println("Application completed");
  }

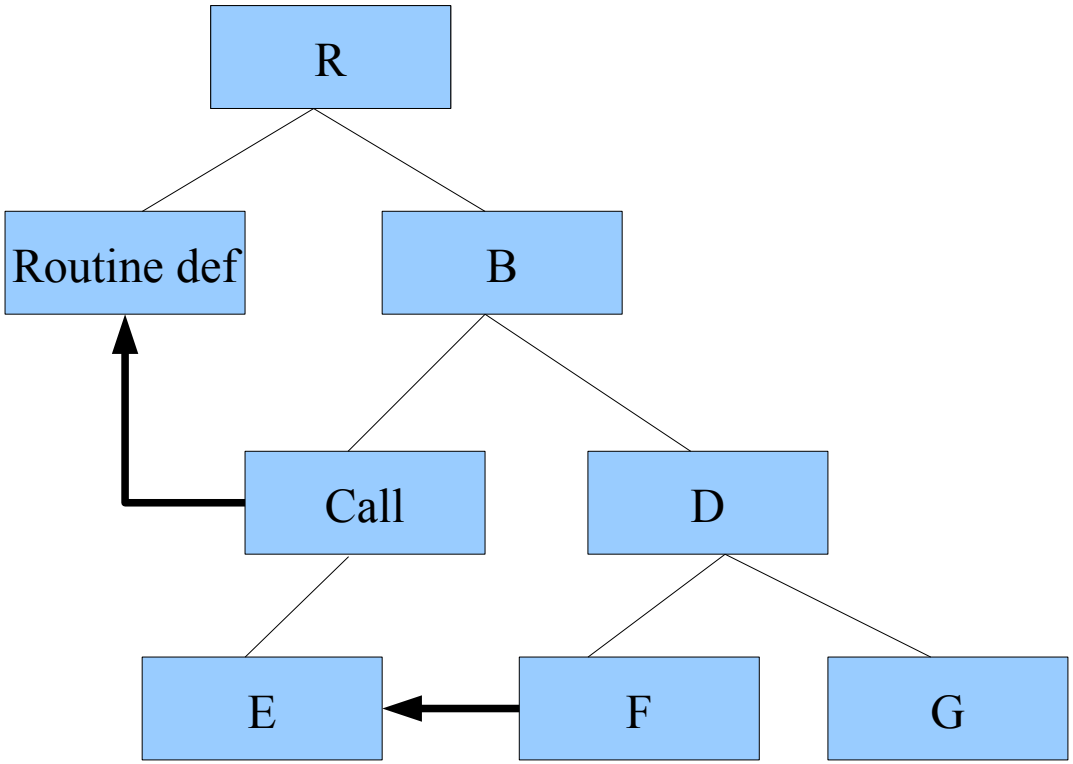
}
```


node
if
Concept = jetbrains.mps.baseLanguage.structure.IfStatement
ifTrue : StatementList
Concept = jetbrains.mps.baseLanguage.structure.StatementList
statement : ExpressionStatement
statement : for
Concept = jetbrains.mps.baseLanguage.structure.ForeachStatement
body : StatementList
Concept = jetbrains.mps.baseLanguage.structure.StatementList
statement : ExpressionStatement
Concept = jetbrains.mps.baseLanguage.structure.ExpressionStatement
expression : DotExpression {void}
Concept = jetbrains.mps.baseLanguage.structure.DotExpression
operand : .<static field> {java.io.PrintStream}
operation : println(String):void {void}
properties
referents
properties
referents
properties
referents
variable : value {string}
iterable : args {string[]}
properties
referents
properties
referents
condition : > {boolean}
ifFalseStatement : {
properties
referents

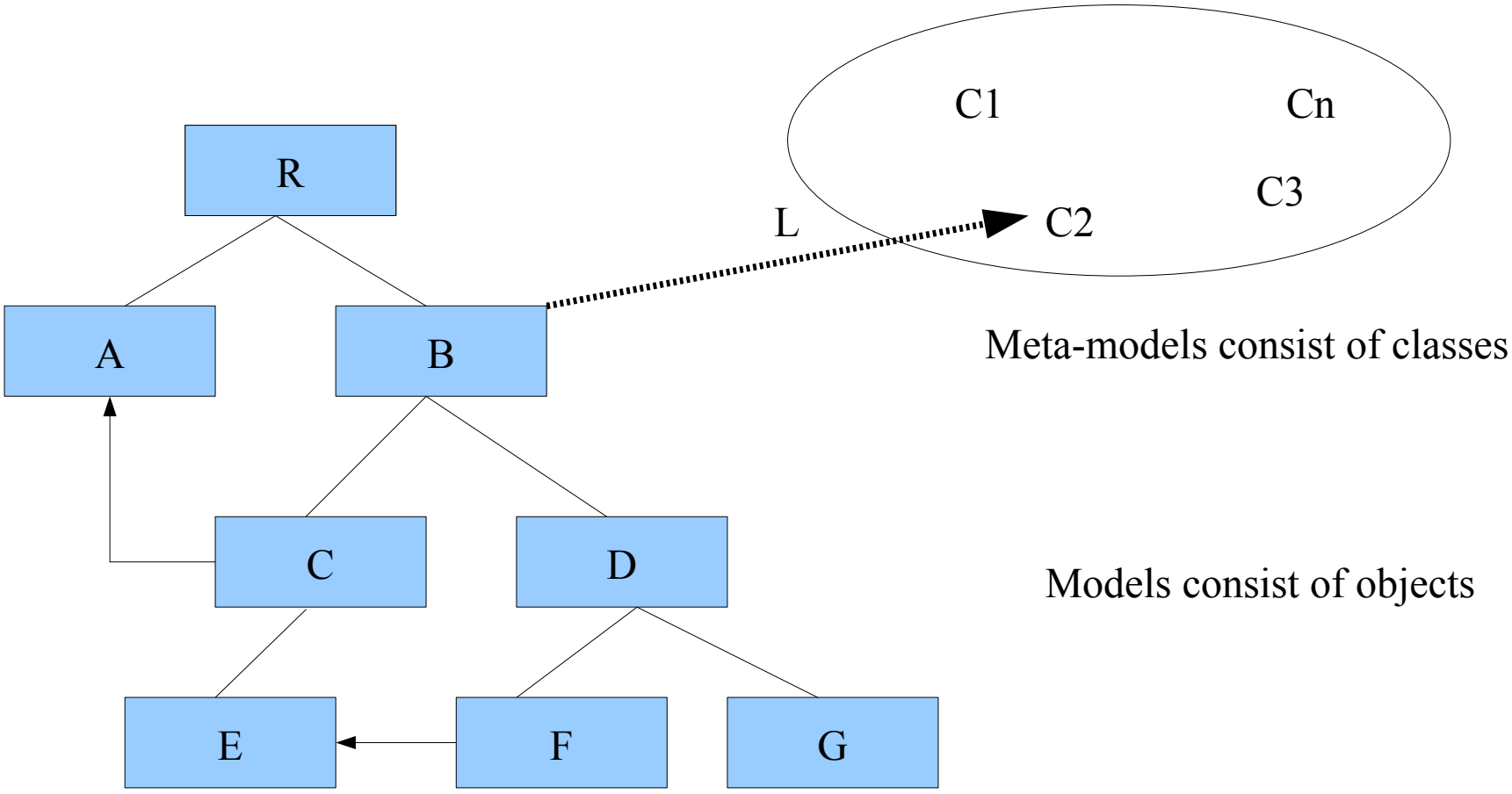
Children have roles



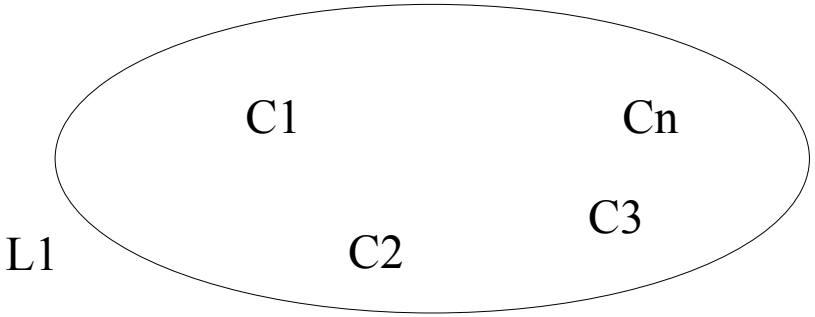
References cross the tree



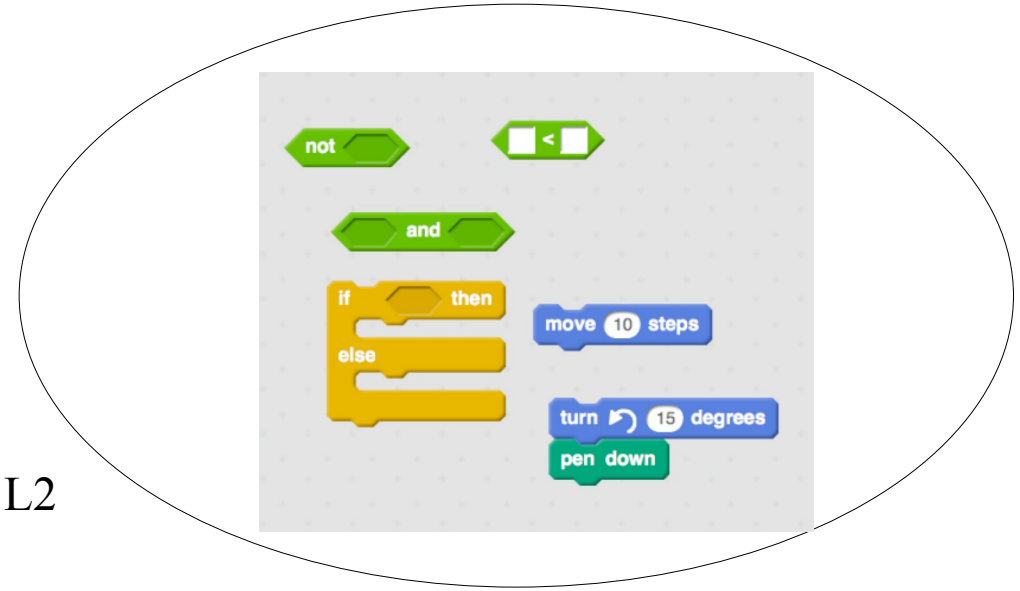
Programs and Languages



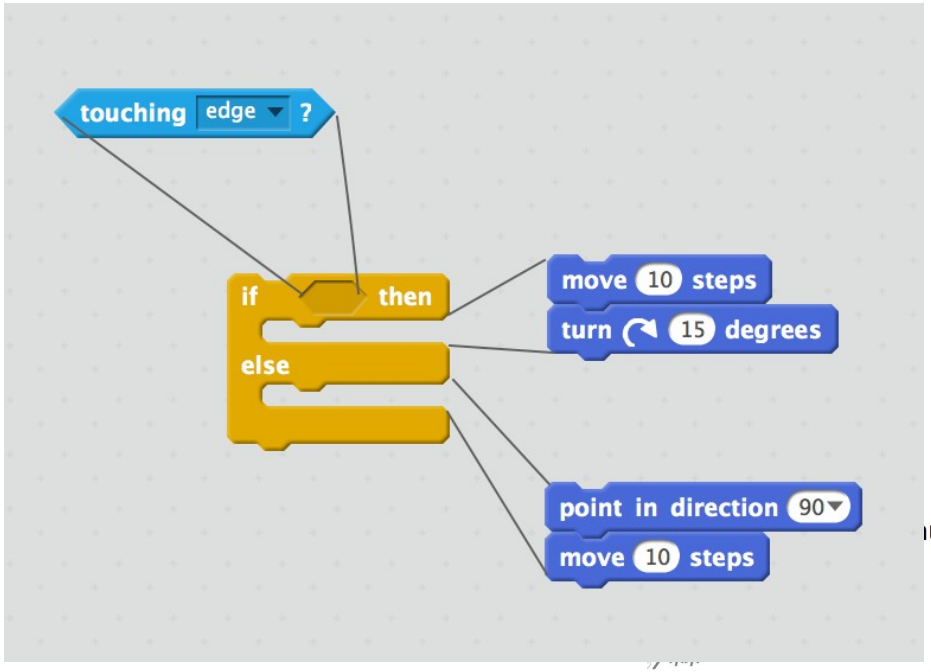
Languages are sets of classes



L1



L2




```
us.groovy.ast.stmt;

 * Represents an if (condition) { ... } else { ... } statement in Groovy
 *
 * @author <a href="mailto:james@coredevelopers.net">James Strachan</a>
 * @version $Revision$
 */
public class IfStatement extends Statement {

    private BooleanExpression booleanExpression;
    private Statement ifBlock;
    private Statement elseBlock;


    public IfStatement(BooleanExpression booleanExpression, Statement ifBlock,
        this.booleanExpression = booleanExpression;
        this.ifBlock = ifBlock;
        this.elseBlock = elseBlock;
    }
```

The Node

```
public class ASTNode {

    private int lineNumber = -1;
    private int columnNumber = -1;
    private int lastLineNumber = -1;
    private int lastColumnNumber = -1;
    private ListHashMap metaDataMap = null;

    public void visit(GroovyCodeVisitor visitor) {
        throw new RuntimeException("No visit() method implemented for class: "
    }
```

```
public class TernaryExpression extends Expression {

    private BooleanExpression booleanExpression;
    private Expression trueExpression;
    private Expression falseExpression;
```

```
public class ElvisOperatorExpression extends TernaryExpression {

    public ElvisOperatorExpression(Expression base, Expression falseExpression) {
        super(getBool(base), base, falseExpression);
    }
```

```
public class ForStatement extends Statement implements LoopingStatement {
    public static final Parameter FOR_LOOP_DUMMY = new Parameter(ClassHelp

    private Parameter variable;
    private Expression collectionExpression;
    private Statement loopBlock;
    private VariableScope scope;
```

```
public class MethodNode extends AnnotatedNode implements Opcodes {

    public static final String SCRIPT_BODY_METHOD_KEY = "org.codeha
    private final String name;
    private int modifiers;
    private boolean syntheticPublic;
    private ClassNode returnType;
    private Parameter[] parameters;
    private boolean hasDefaultValue = false;
    private Statement code;
    private boolean dynamicReturnType;
    private VariableScope variableScope;
    private final ClassNode[] exceptions;
    private final boolean staticConstructor;

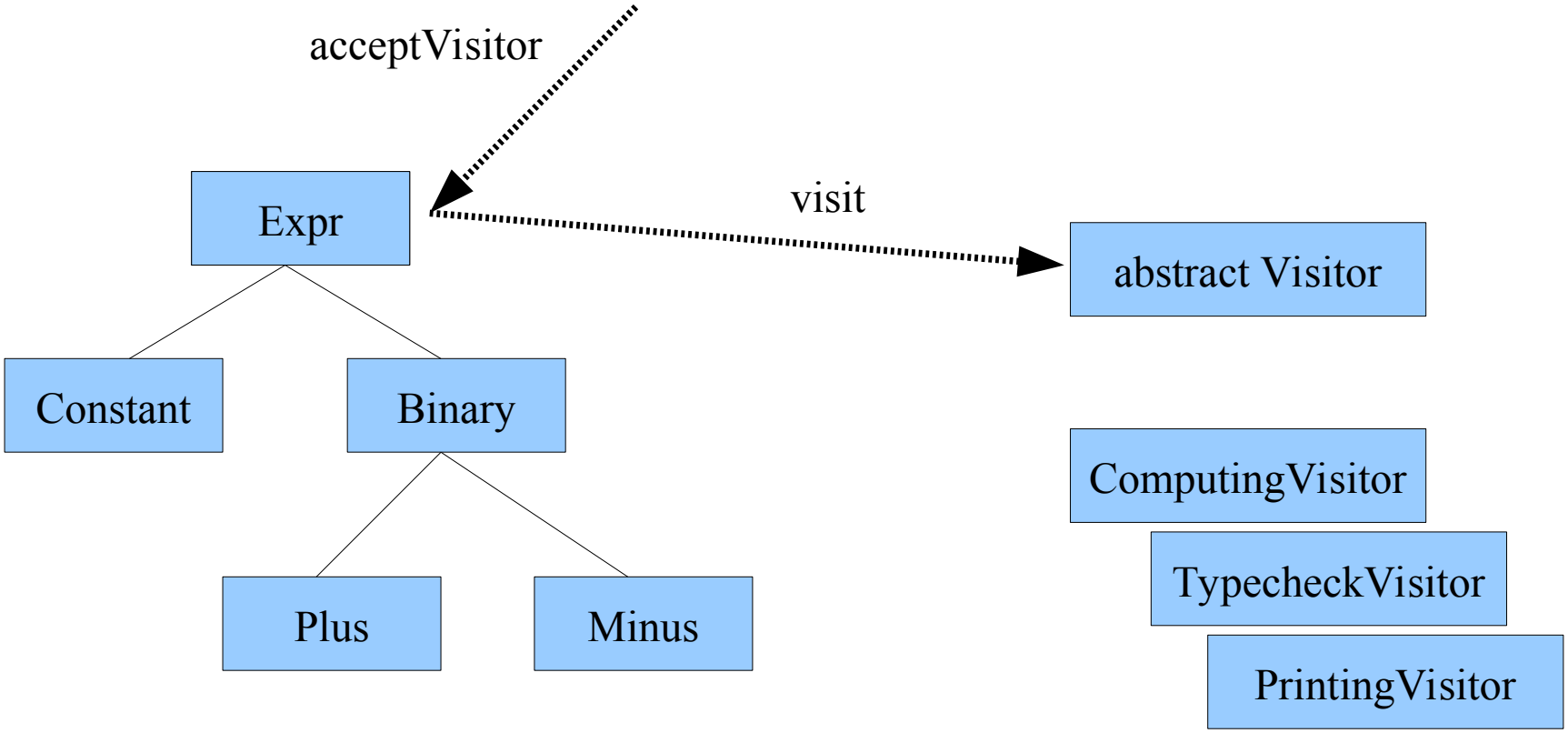
    // type spec for generics
    private GenericsType[] genericsTypes = null;
    private boolean hasDefault;

    // cached data
    String typeDescriptor;
```

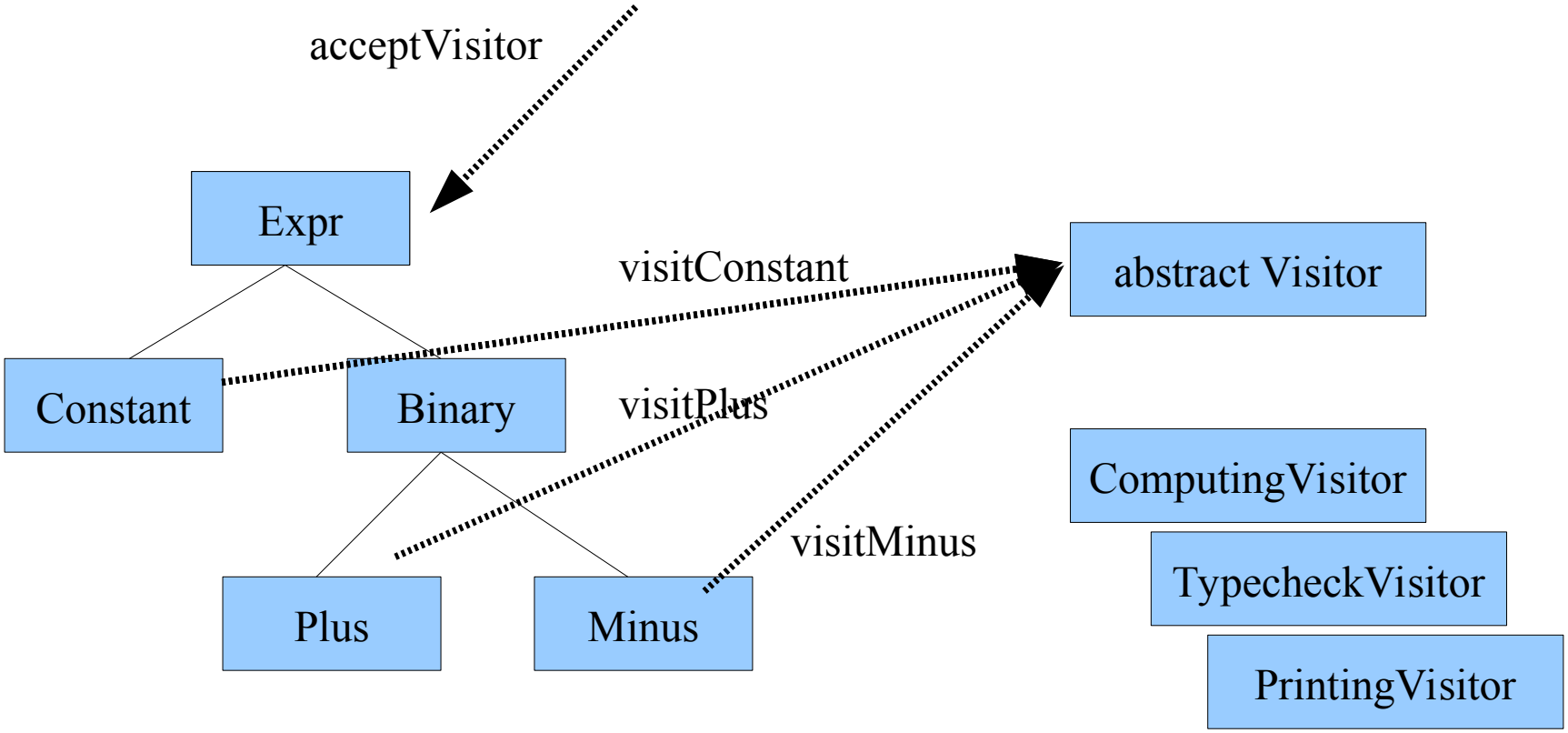
Visitor pattern

Separate class hierarchies from their processing

Visitor pattern



Visitor pattern



There's no life without trees

- Extending IDEs
- External DSLs
- Internal DSLs in modern languages
  - Groovy AST transformations
  - Scala macros
  - Lisp, Clojure macros
  - Java Lombok library

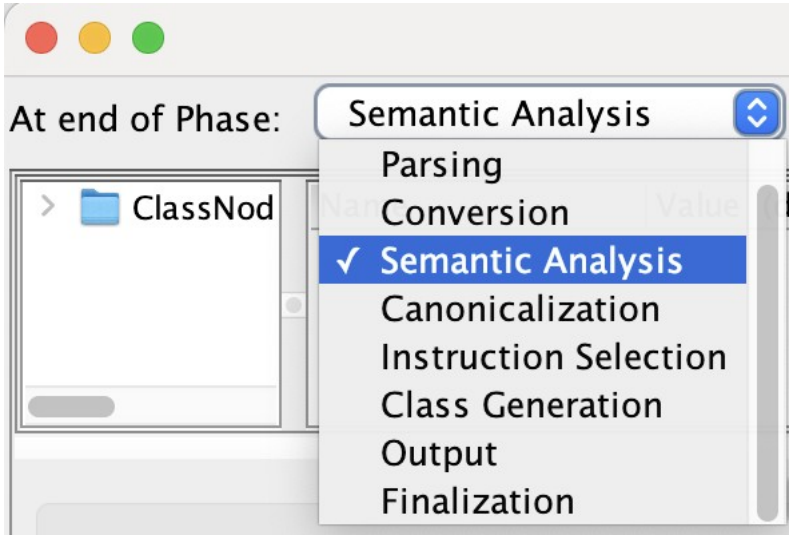
AST Transformations

```
class Registrations {
    @Delegate List items = []
}

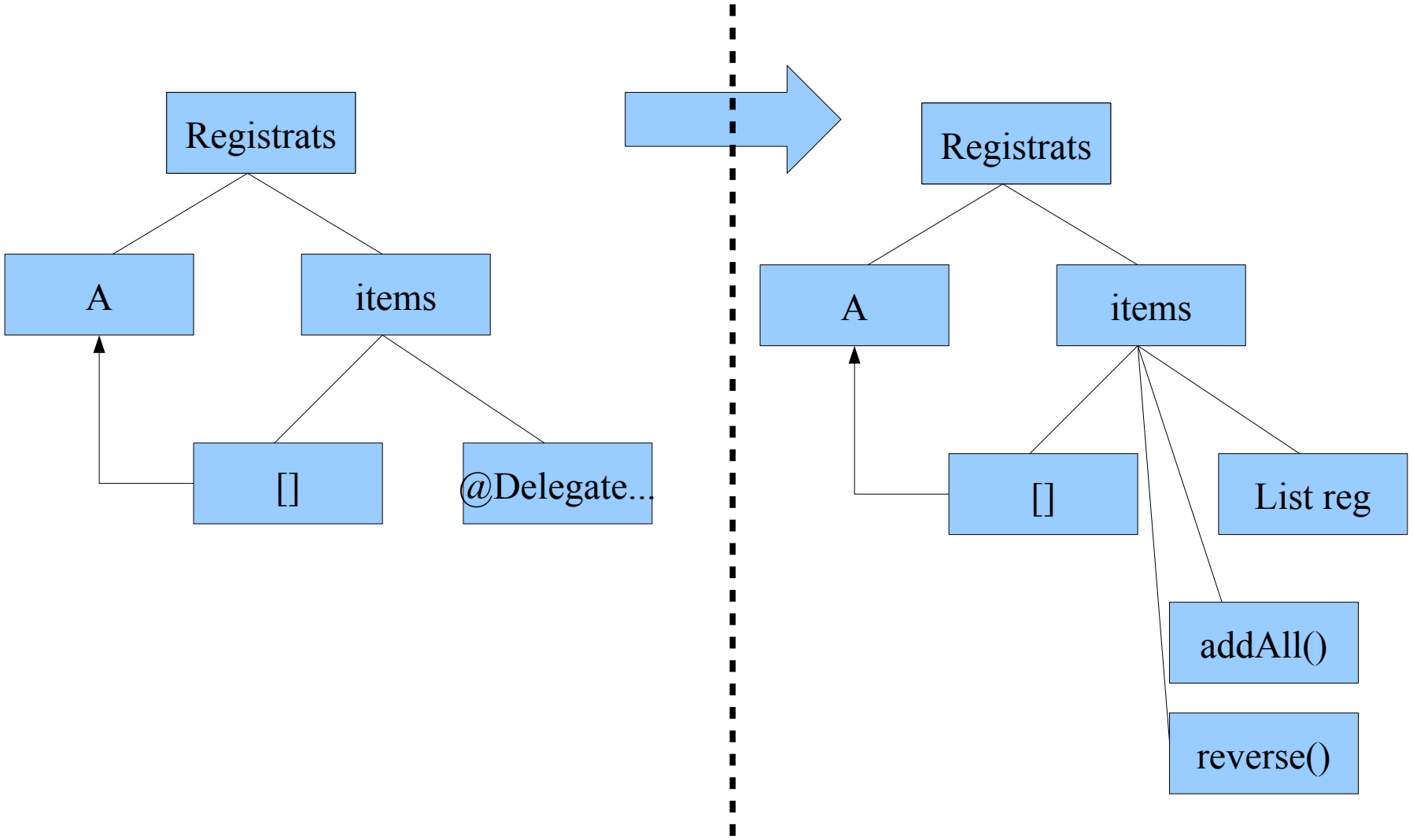
def regs = new Registrations()
regs.addAll(["Joe", "Dave"])
assert ["Dave", "Joe"] == regs.reverse()
```

AST Transformations

- Hook into the compiler
- Act as visitors on the AST
- Apply at the end of any of the phases



Ast transformation



Out-of-the-box transforms

@Delegate, @Immutable, @Singleton

@Lazy

@TupleConstructor

@InheritConstructors

@Canonical

@ToString

@EqualsAndHashCode

Out-of-the-box transforms

@Log, @Log4j, @Commons

@Synchronized

@WithReadLock

@WithWriteLock

@AutoClone, @AutoExternalize

… and many more

Type-checking/Static

@TypeChecked, *@CompileStatic*

```
@TypeChecked
String test(Object val) {
    if (val instanceof String) {
        val.toUpperCase()
    } else if (val instanceof Number) {
        val.intValue() * 2
    }
}
```

Creating AST Transformations

```
@Retention(RetentionPolicy.SOURCE)
@Target([ElementType.METHOD])
@GroovyASTTransformationClass("RequiresTransformation")
public @interface Requires {
    String value() default "true";
}

@GroovyASTTransformation(phase = SEMANTIC_ANALYSIS)
public class RequiresTransformation implements ASTTransformation {

    public void visit(ASTNode[] astNodes, SourceUnit source) {
        ...
    }
}
```

Parse code into AST

```
def createStatements(String clause) {
    def statements = """
        if(!($clause)) {
            throw new Exception('Precondition violated: {$clause}')
        }
    """

    AstBuilder ab = new AstBuilder()
    List<ASTNode> res = ab.buildFromString(CompilePhase.SEMANTIC_ANALYSIS, statements)
    BlockStatement bs = res[0]
    return bs
}
```

Report compilation errors

```
public void addError(String msg, ASTNode expr, SourceUnit source) {
    int line = expr.lineNumber
    int col = expr.columnNumber
    SyntaxException se = new SyntaxException(msg + '\n', line, col)
    SyntaxErrorMessage sem = new SyntaxErrorMessage(se, source)
    source.errorCollector.addErrorAndContinue(sem)
}
```

Ways to create AST

1. Build from a string
2. Using AST classes directly
3. Use factory methods
4. Use the specification builder
    - AstSpecificationCompiler

Build from a string

Parses code to the AST form corresponsing to the specified compilation phase

```
.buildFromString ('''
                    Integer.parseInt(param)
''')
```

Using AST classes directly

```
@GroovyASTTransformation(phase = CompilePhase.INSTRUCTION_SELECTION)
class AddMethodASTTransformation extends AbstractASTTransformation {
    @Override
    void visit(ASTNode[] nodes, SourceUnit source) {
        ClassNode classNode = (ClassNode) nodes[1]

        ReturnStatement code =
                new ReturnStatement(                                    (1)
                        new ConstantExpression("42"))                   (2)

        MethodNode methodNode =
                new MethodNode(
                        "getMessage",
                        ACC_PUBLIC,
                        ClassHelper.make(String),
                        [] as Parameter[],
                        [] as ClassNode[],
                        code)                                           (3)

        classNode.addMethod(methodNode)                                 (4)
    }
}
```

Use factory methods

// Constant expression

GeneralUtils.constX("Hello")

// Assignment

GeneralUtils.assignX(varX("x"), constX(42))

//Method call – *c.m(a)*

GeneralUtils.returnS(callX(c, "m", varX('a')))

AST Specification Builder

```
.buildFromSpec {
    method('convertToNumber', ACC_PUBLIC, Integer) {
            parameters { parameter 'parameter': String.class }
            exceptions {}
            block {
                returnStatement {
                    staticMethodCall(Integer, "parseInt") {
                        argumentList {
                            variable "parameter"
                        }
        }  }  }  }  }
```

AstSpecificationCompiler

The delegate for the *buildFromSpec()* closures

Summary



AST transformations – use and creation

References

http://groovy-lang.org

https://projectlombok.org/